

Terraform pro inženýry (Infrastructure as Code)

Cíl dokumentu

Tento dokument shrnuje Terraform z pohledu inženýra, který:

- chce popsat infrastrukturu (cloud/on-prem) jako kód,
- nasazuje a modifikuje prostředí konzistentně napříč regiony a účty,
- používá Terraform v týmu (state, moduly, CI/CD).

1. Co je Terraform

Terraform je nástroj pro Infrastructure as Code (IaC), který umožňuje:

- deklarativně popsat cílový stav infrastruktury v HCL (HashiCorp Configuration Language),
- plánovat změny (`terraform plan`),
- aplikovat změny (`terraform apply`) bezpečně a postupně,
- verzovat infrastrukturu v Git repozitáři.

Terraform pracuje nad API poskytovatelů (cloud, SaaS, Kubernetes, Helm, GitHub,...),

- každý "svět" má svého **provider**,
- provider implementuje resource typy a data sources.

2. Základní koncepty

2.1 Provider

Provider říká, s jakou platformou Terraform komunikuje:

- AWS, Azure, GCP,
- Kubernetes, Helm,
- GitHub, Datadog, Cloudflare, atd.

V konfiguraci specifikuješ:

- typ provideru (např. `aws`),
- verzi provideru,
- parametry (region, credentials,...).

2.2 Resource

Resource je objekt, který Terraform vytváří / modifikuje / maže.

Příklady:

- `aws_instance`, `aws_s3_bucket`, `google_compute_network`,
- `kubernetes_deployment`, `helm_release`.

Resource blok definuje:

- **typ** (např. `aws_instance`),
- **jmenný identifikátor** (např. `web`),
- **atributy** (parametry) – název, velikost, tagy, atd.

2.3 Data source

Data source je "read-only" pohled do existujícího světa.

Příklady:

- `data "aws_ami"` pro získání ID posledního AMI,
- `data "aws_vpc"` pro lookup existující VPC,
- `data "kubernetes_secret"` pro získání tajemství.

Data source se používají třeba pro:

- kombinaci existující infrastruktury s novou,
- automatizaci (např. vždy použij poslední image).

2.4 State

Terraform state je centrální soubor (typicky `terraform.tfstate`), který:

- reprezentuje aktuální realitu – jaké resource Terraform vytvořil a jak vypadají,
- slouží k porovnání při `plan` (stav v cloudu vs. konfigurace vs. state),
- obsahuje citlivá data (ID, hesla, outputy) – musí být bezpečně uloženy.

State může být:

- lokální (pro experimenty, single-user),
- vzdálený (S3 + DynamoDB lock, GCS, Terraform Cloud, atd.) – to je nutnost pro týmovou práci.

2.5 Moduly

Module je běžný Terraform adresář s konfigurací, který může být znovu použit:

- root modul – hlavní konfigurace v konkrétním workspace,
- child moduly – menší kousky infrastruktury (např. VPC modul, aplikace modul).

Výhody modulů:

- sdílení best practices,
- menší opakování kódu,
- jednodušší údržba – opraviš modul a znovu ho použiješ všude.

3. Terraform workflow

Základní cyklus práce s Terraformem:

1. **Write** – napíšeš změny do HCL souborů (.tf).
2. **Plan** – terraform plan ukáže, co se stane (ADD/CHANGE/DESTROY).
3. **Apply** – terraform apply změny provede.

Terraform vyhodnocuje závislosti mezi resources a:

- vytváří resources v paralelních skupinách, pokud to jde,
- respektuje dependency (např. VPC → subnety → instance → DNS).

4. Struktura projektu

Typický layout:

- main.tf – hlavní konfigurace,
- variables.tf – definice vstupních proměnných,
- outputs.tf – definice výstupů,
- /modules – lokální moduly,
- /envs – různé konfigurace pro prostředí.

Alternativně:

- oddělené repozitáře per prostředí,
- monorepo s více složkami pro různé oblasti infrastruktury.

5. Proměnné, outputs a locals

5.1 Variables

variable bloky definují vstupy:

- mohou mít default hodnotu,
- mohou být citlivé (sensitive),
- mohou mít validace.

Hodnoty proměnných dodáváš:

- přes *.tfvars soubory,
- přes CLI flagy (-var),
- přes environment proměnné,
- přes CI/CD secret store.

5.2 Outputs

output bloky definují, co Terraform vrátí po apply:

- např. URL load balanceru, ID resource,
- outputs mohou být použity v jiných projektech (např. jako input pro další Terraform).

5.3 Locals

locals umožňují:

- definovat pomocné výrazy a common logiku,
- například složené názvy resource (local.name),
- transformace seznamů / map.

6. Backend a remote state

Backend definuje, kde se ukládá state.

Příklady backendů:

- AWS S3 + DynamoDB lock,
- Google Cloud Storage,
- Terraform Cloud / Enterprise,
- Consul.

Výhody remote state:

- sdílený state pro tým,
- locking – zabrání paralelním apply,
- zálohování a audit.

7. Moduly v praxi

Použití modulů:

- modul VPC (subnety, routy, security groups),
- modul pro aplikaci (ALB + autoscaling + security group + target group),
- modul pro logování / monitoring.

Moduly mohou být lokální nebo vzdálené (Terraform Registry, Git repo).

Důležité je:

- modul mít verzovaný (tag v Gitu, verze v registry),
- modul dobře zdokumentovat (inputs, outputs, příklad použití),
- udržovat semantické verzování (major změny jako breaking changes).

8. Prostředí (workspaces a separace)

Možnosti, jak oddělit dev/stage/prod:

- samostatné projekty/složky per prostředí,
- workspaces v Terraformu (pro jednoduché scénáře),
- kombinace – workspace pro drobné varianty, jiný projekt pro zásadně odlišnou architekturu.

Oddělení by mělo být jasné a bezpečné – aby např. prod prostředí nebylo ohroženo chybou v dev.

9. Integrace s Kubernetes a Helm

Terraform má poskytovatele:

- `kubernetes` – pro tvorbu K8s objektů,
- `helm` – pro správu Helm release.

To umožní:

- vytvořit infrastrukturu (VPC, cluster, databáze),
- a poté nasadit aplikace do clusteru pomocí Helm release – vše v jednom Terraform projektu.

Je třeba dávat pozor na:

- pořadí (cluster musí vzniknout, než Terraform spustí `helm_release`),
- credentials pro přístup do clusteru,
- správu verzí chartů.

10. Best practices

- verzuj Terraform kód v Gitu, používej code review,
- nikdy nenechávej citlivý lokální state – používej remote backend,
- používej moduly pro opakující se patterny,
- validuj konfiguraci (pre-commit hooks, terraform fmt, terraform validate),
- loguj plan a apply výstupy (audit),
- při větších změnách dělej postupné apply – odděl infrastrukturu do více modulů/projektů.